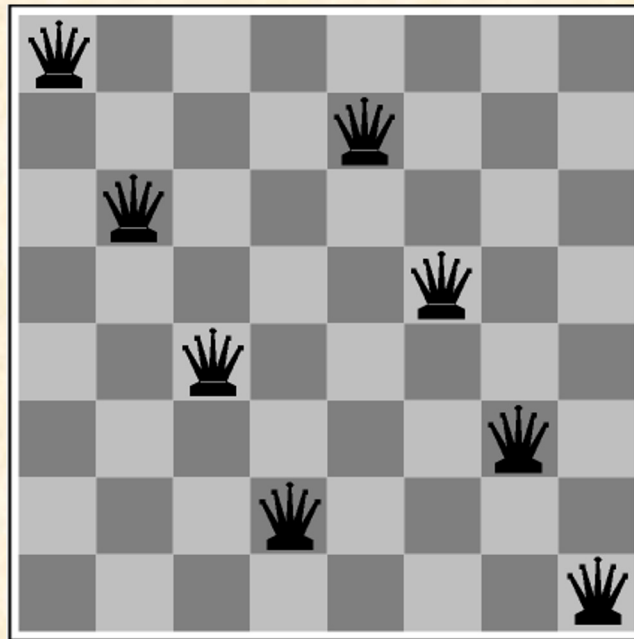


Lecture 22

Backtracking & Branch and Bound Algorithms



Data Structures & Algorithms

Outline

- Review
 - Constraint Satisfaction
 - Optimization
- Backtracking
 - General Form
 - n Queens
- Branch and Bound
 - Traveling salesperson problem

Backtracking



Data Structures & Algorithms

Types of Algorithm Problems 1/3

- Constraint satisfaction problems
 - Can we satisfy all given constraints?
 - If yes, how do we satisfy them?
 - Need a specific solution
 - May have more than one solution
 - Examples: sorting, puzzles, GRE/analytical
- Optimization problems
 - Must satisfy all constraints (can we?) and
 - Must minimize an objective function subject to those constraints

Types of Algorithm Problems 2/3

- Constraint satisfaction problems
 - Go over all possible solutions
 - Does a given input combination satisfy all constraints?
 - *Can stop when a satisfying solution is found*
- Optimization problems
 - Similar, except we also need to compute the objective function every time
 - *Stopping early = possible non-optimal solution*

Types of Algorithm Problems 3/3

- Constraint satisfaction problems
 - Can rely on *Backtracking algorithms*
- Optimization problems
 - Can rely on *Branch and Bound algorithms*

For particular problems, there may be much more efficient approaches, but think of these as a fallback to a more sophisticated version of a brute-force approach.

General Form: Backtracking

```
Algorithm checknode(node v)
  if (promising(v))
    if (solution(v))
      write solution*
    else
      for each node u adjacent to v
        checknode(u)
```

* Can exit here if only the existence of a solution is needed

Backtracking Components

solution(v)

- Check 'depth' of solution (constraint satisfaction)

promising(v)

- Different for each application

checknode(v)

- Called only if partial solution is both promising and not a solution

An Alternate Form: Backtracking

```
Algorithm checknode(node v)
    if (solution(v))
        write solution*
    else
        for each node u adjacent to v
            if (promising(u))
                checknode(u)
```

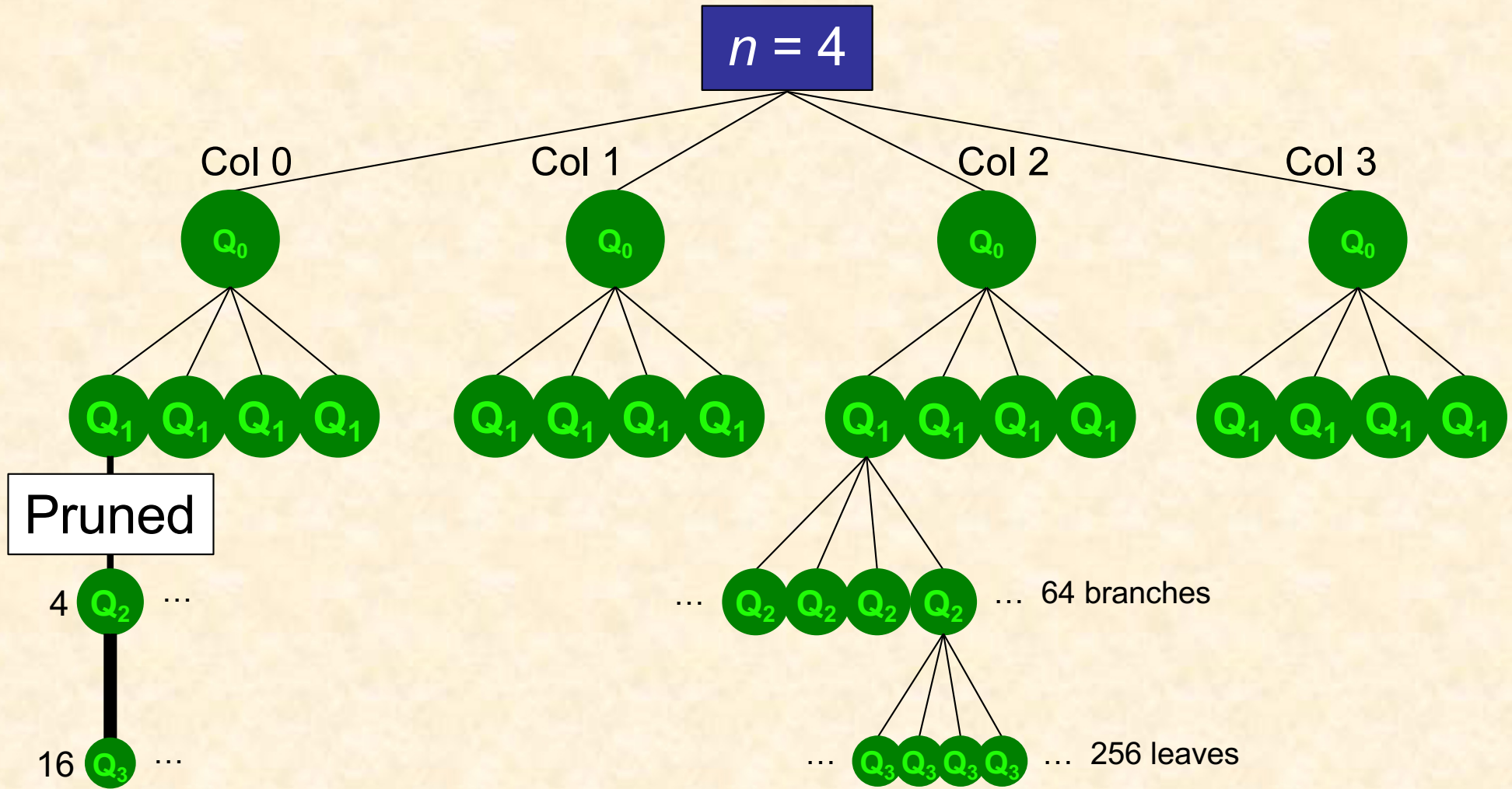
* Can exit here if only the existence of a solution is needed

Backtracking Example: n Queens

Can n queens be placed on an $n \times n$ board so that no queens are threatened?

- $n = 1$: 1 queen, 1 x 1 board
- $n = 2$: 2 queens, 2 x 2 board
- $n = 3$: 3 queens, 3 x 3 board
- $n = 4$: 4 queens, 4 x 4 board
- $n = 5$: 5 queens, 5 x 5 board
- ...

Search Tree: n Queens



4 Queens Search

For 4 Queens

- Entire search tree has **256** leaves
- Backtracking enables searching of **19** branches before finding first solution
- Promising:
 - May lead to solution
- Not promising:
 - Will never lead to solution
 - Therefore should be pruned

4 Queens Branches

A Q	B Q	C Q	D
E Q	F Q	G Q	H Q
I Q	J Q	K Q	L Q
M Q	N Q	O Q	P Q

Branches Searched

1. **A**->E = vert. threat
2. A->F = diag. threat
3. **A**->**G**->I = vert. threat
4. A->G->J = diag. threat
5. A->G->K = 2 threats
6. A->G->L = diag. threat
7. **A**->**H**->I = vert. threat
8. **A**->**H**->**J**->M = 2 threats
9. A->H->J->N = 2 threats
10. A->H->J->O = diag. threat
11. A->H->J->P = 2 threats
12. A->H->K = 2 threats
13. A->H->L = vert. threat
14. **B**->E = diag. threat
15. B->F = vert. threat
16. B->G = diag. threat
17. **B**->**H**->**I**->M = vert. threat
18. B->H->I->N = 3 threats
19. **B**->**H**->**I**->**O** = **SOLUTION**

Backtracking Functions: n Queens 1/2

solution(v)

- Check 'depth' of solution (constraint satisfaction)
- Placed queen on each row
- That is, depth = N

checknode(v)

- Called only if promising and not solution
- Recursive call to all positions (columns) of queen within row

Backtracking Functions: n Queens 2/2

promising(row, col)

- Called for each node of the search tree
- Assume data structures that can tell you if:
 - `column[col]` *// is column 'col' available*
 - `leftDiagonal[x]` *// is upper-left to lower-right diagonal available*
 - `rightDiagonal[y]` *// is upper-right to lower-left diagonal available*
- NOT promising if any of these are unavailable
 - We'll see what 'x' and 'y' are soon...

8 Queens: Search Space

- Brute force checks about 4.43×10^9 possibilities (64 choose 8), including many ridiculous board configurations
- Even with sensible choices (1 queen per row), the search space is still fairly large:
 - **16,777,216** possibilities (8^8)
 - **92** solutions
- How can the search space be further reduced?

Summary: Backtracking

- Backtracking allows pruning of branches that are not promising
- All backtracking algorithms have a similar form
- Often, most difficult part is determining nature of **promising()**

Backtracking



Data Structures & Algorithms

Branch and Bound



Data Structures & Algorithms

Types of Algorithm Problems 1/3

- Constraint satisfaction problems
 - Can we satisfy all given constraints?
 - If yes, how do we satisfy them?
 - Need a specific solution
 - May have more than one solution
 - Examples: sorting, puzzles, GRE/analytical
- Optimization problems
 - Must satisfy all constraints (can we?) and
 - Must minimize an objective function subject to those constraints

Types of Algorithm Problems 2/3

- Constraint satisfaction problems
 - Go over all possible solutions
 - Does a given input combination satisfy all constraints?
 - *Can stop when a satisfying solution is found*
- Optimization problems
 - Similar, except we also need to compute the objective function every time
 - *Stopping early = possible non-optimal solution*

Types of Algorithm Problems 3/3

- Constraint satisfaction problems
 - Can rely on *Backtracking algorithms*
- Optimization problems
 - Can rely on *Branch and Bound algorithms*

For particular problems, there may be much more efficient approaches, but think of these as a fallback to a more sophisticated version of a brute-force approach.

Branch-and-Bound, a.k.a. B&B

- The idea of backtracking **extended** to *optimization* problems
- You are minimizing a function with this useful property:
 - A partial solution is pruned if its cost $\geq$ cost of best known complete solution
 - e.g., the length of a path or tour
- If the cost of a partial solution is too big **drop this partial solution**

General Form: Branch & Bound

Algorithm checknode(Node v, Best currBest)

Node u

if (promising(v, currBest))

if (solution(v)) then

update(currBest)

else

for each child u of v

checknode(u, currBest)

return currBest

Branch & Bound Components 1

solution()

- Check 'depth' of solution (constraint satisfaction)

update()

- If new solution better than current solution, then update (optimization)

checknode()

- Called only if promising and not solution

Branch & Bound Components 2

lowerbound()

- Estimate of solution based upon
 - Cost so far, plus
 - Underestimate of cost remaining (aka bound)

promising()

- Different for each application, but must return true when **lowerbound() < currBest**
- A return of false is what causes pruning ($\geq$)

The Key to B&B is the **Bound**

- The efficiency of B&B is based on “bounding away” (aka “pruning”) unpromising partial solutions
- The earlier you know a solution is not promising, the less time you spend on it
- The more accurately you can compute partial costs, the earlier you can prune
- Sometimes it’s worth spending extra effort to compute better bounds

Minimizing With B&B

- Start with an infinity bound
- Find first complete solution – use its cost as an upper bound to prune the rest of the search
- Measure each partial solution and calculate a lower bound estimate needed to complete the solution
- Prune partial solutions whose lower bounds are $\geq$ the current upper bound
- If another complete solution yields a lower cost – that will be the new upper bound
- When search is done, the current upper bound will be a minimal solution

Maximizing With B&B

- Start with a negative one bound
- Find first complete solution – use its cost as a lower bound to prune the rest of the search
- Measure each partial solution and calculate an upper bound estimate needed to complete the solution
- Prune partial solutions whose upper bounds are $\leq$ the current lower bound
- If another complete solution yields a larger value – that will be the new lower bound
- When search is done, the current lower bound will be a maximal solution

Summary: Branch and Bound

- Method to prune search space for optimization problems
- Need to keep current best solution
- Measure partial solutions and combine with **optimistic** estimates of their completions
- If estimate is not an improvement, actual cannot be either, so prune

Branch and Bound



Data Structures & Algorithms

Traveling Salesperson Problem (TSP)



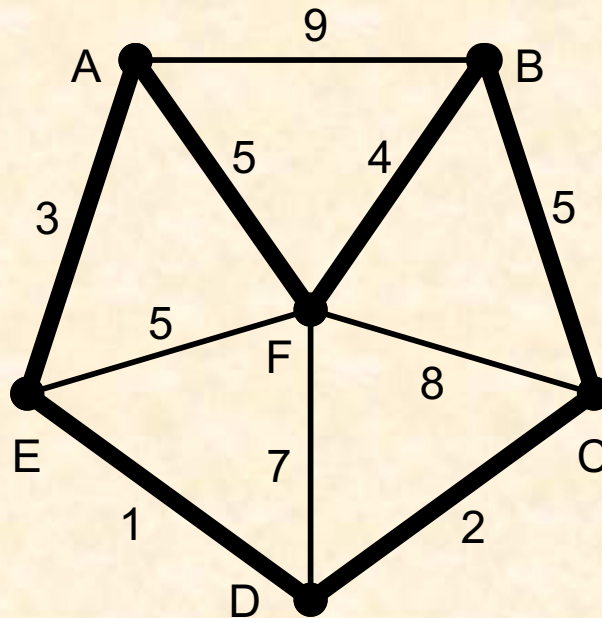
Data Structures & Algorithms

TSP Defined

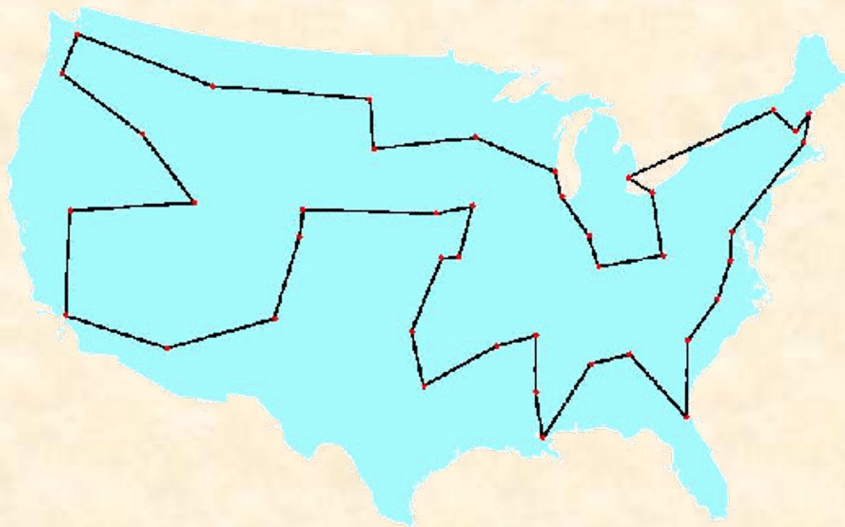
- Hamiltonian Cycle
 - Definition: Given a graph $G = (V, E)$, find a cycle that traverses each node exactly once
 - No vertex may appear twice, except the first/last
 - Constraint satisfaction problem
- Traveling Salesperson Problem
 - Definition: Hamiltonian cycle with least weight
 - Optimization problem

TSP Illustrated

Find tour of minimum length starting and ending in same city and visiting every city exactly once



TSP: (NP) Hard Problem!



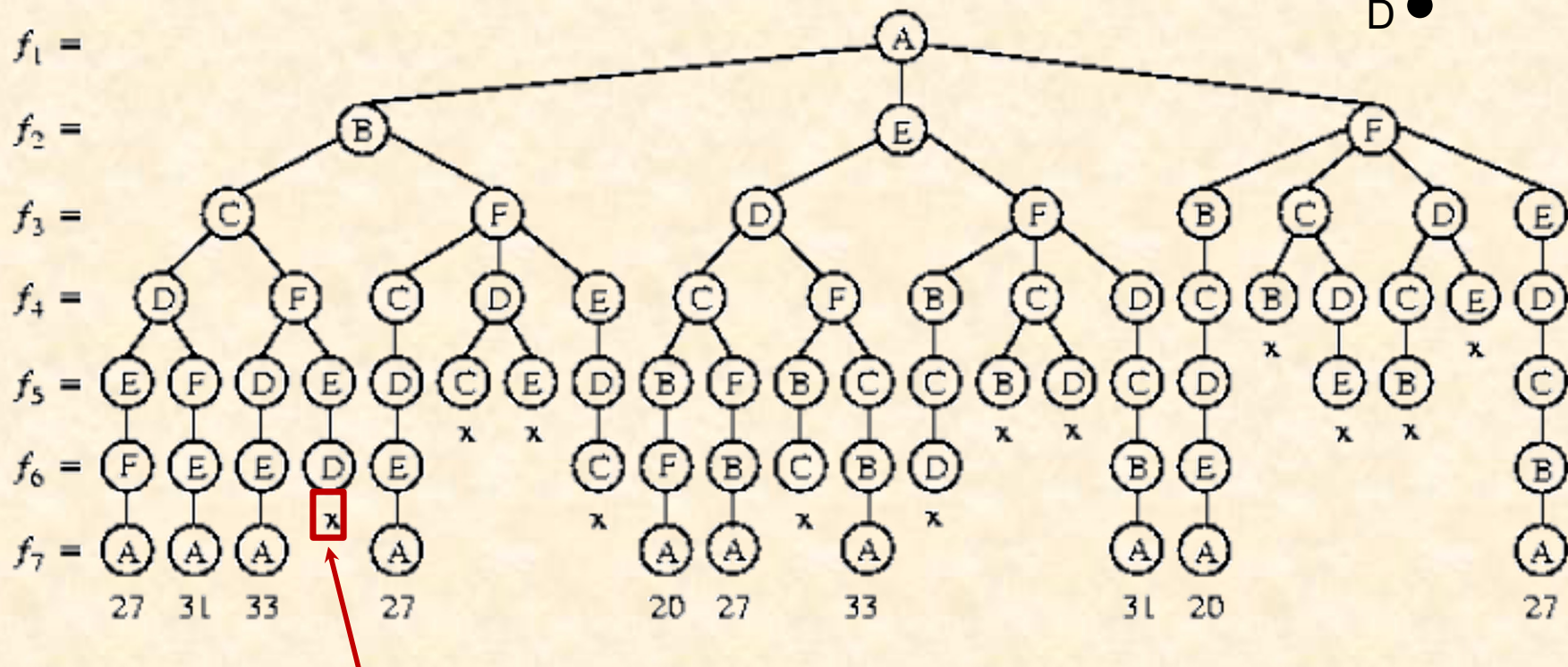
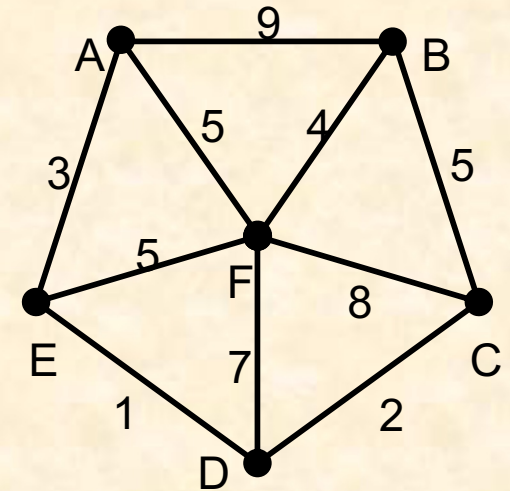
1954: $n = 49$



2004: $n = 24,978$

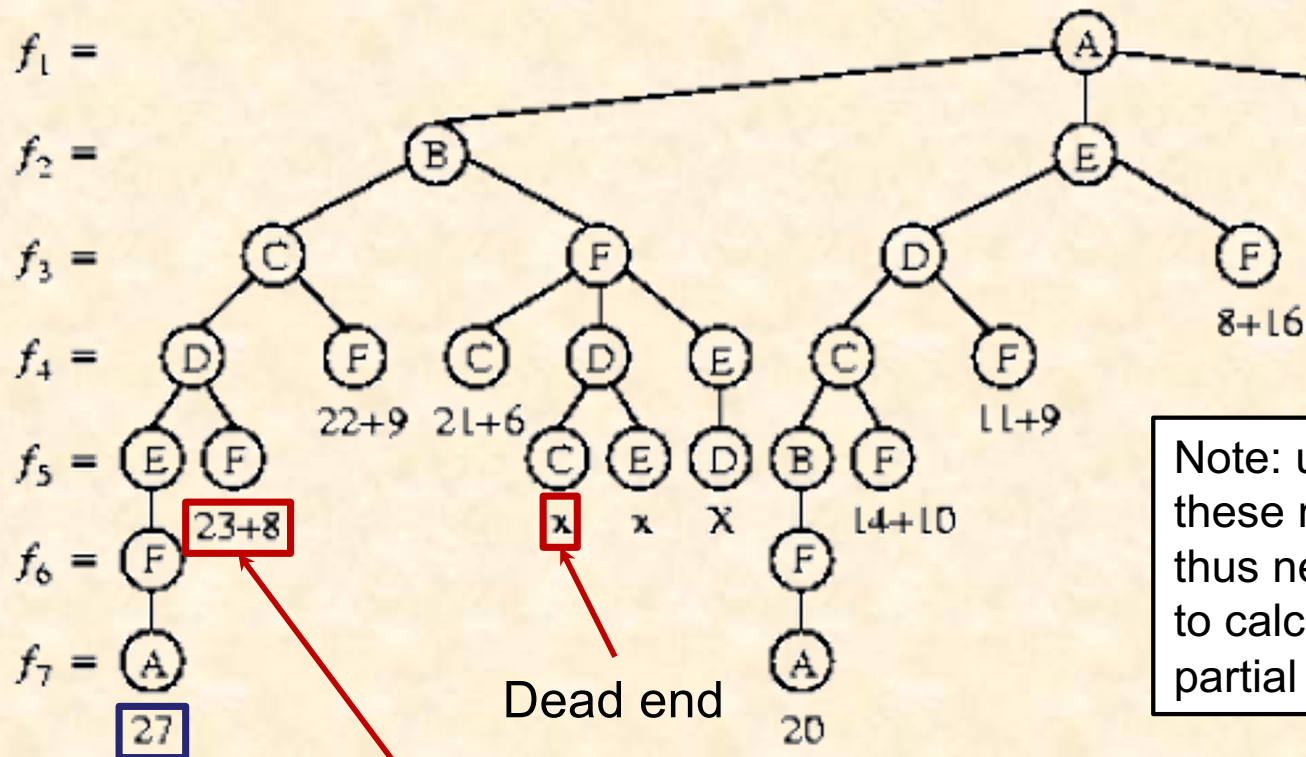
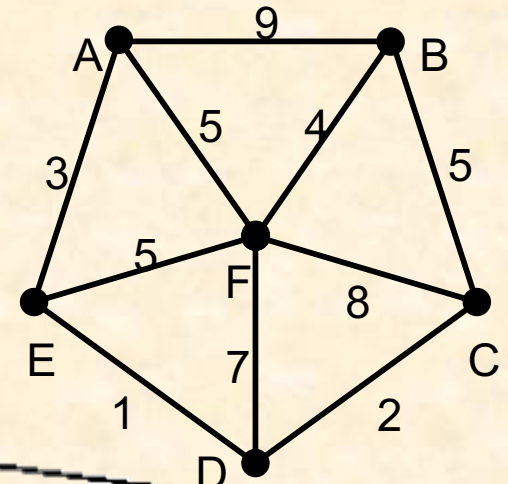
<http://www.math.uwaterloo.ca/tsp/sweden/index.html>

TSP with Backtracking



Dead end in the graph = unpromising partial solution
(all adjacent vertices are already visited)

Advantage of TSP with B&B



Note: usually you don't have these min-cost values and thus need to design a function to calculate a bound given a partial solution.

Best
solution
so far

Minimum cost if we complete a cycle from this partial solution... if cost is > 27 this partial solution is not promising

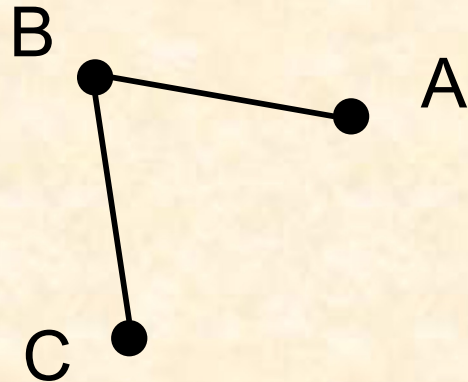
Bounding Function 1

- Estimate must be $\leq$ reality
- The bounding function must have complexity better than just continuing TSP for the k vertices not yet visited:
 - For instance, $O(k^2)$ is better than $O(k!)$ for most values of k
- What method can we use to find the lowest cost way to connect k vertices together in $O(k^2)$ time?

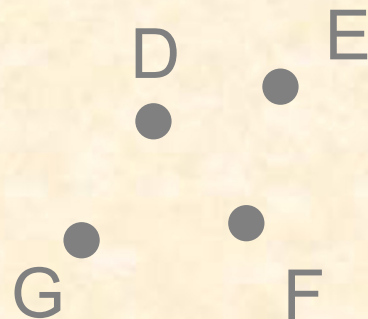
Bounding Function 2

- Some vertices are connected so far, some vertices are not
- For ONLY the unvisited vertices, connect them together with lowest possible cost
- Then connect the visited vertices to the unvisited
- Yes, this function considers solutions that violate constraints, but it's a LOWER bound so it's OK

Partial TSP Example



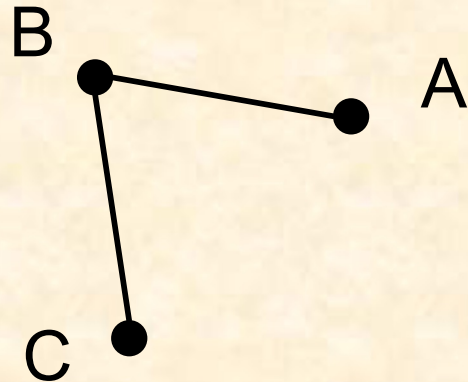
Current path: {A, B, C, F, E, G, D}
permLength = 3



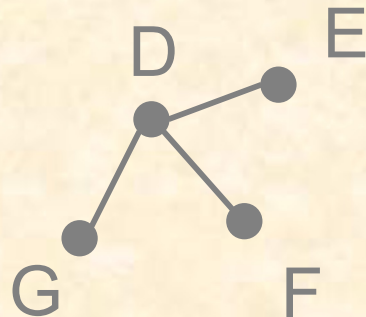
What's the best way
to connect D, E, F, and G
to each other?

Unvisited vertices: F, E, G, and D

Connect Unvisited Nodes Together



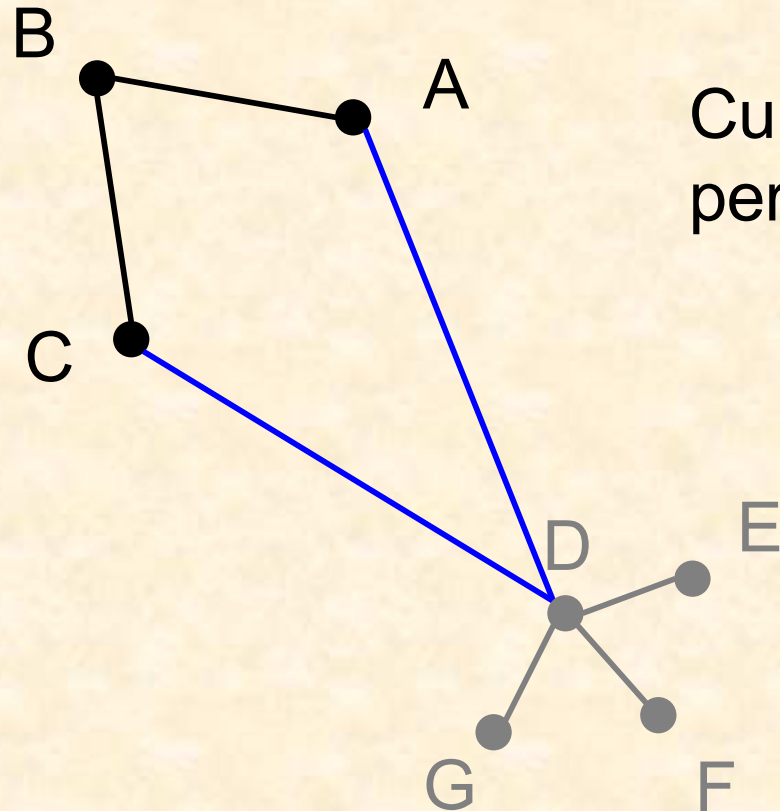
Current path: {A, B, C, F, E, G, D}
permLength = 3



How many edges are we missing? A full TSP tour would have V edges (7), currently we have 5...

Unvisited vertices: F, E, G, and D

Connect Partial Tour to Unvisited



Current path: {A, B, C, F, E, G, D}
permLength = 3

Connect from A-B-C to
D-E-F-G in the best,
cheapest, fastest way
possible

Unvisited vertices: F, E, G, and D

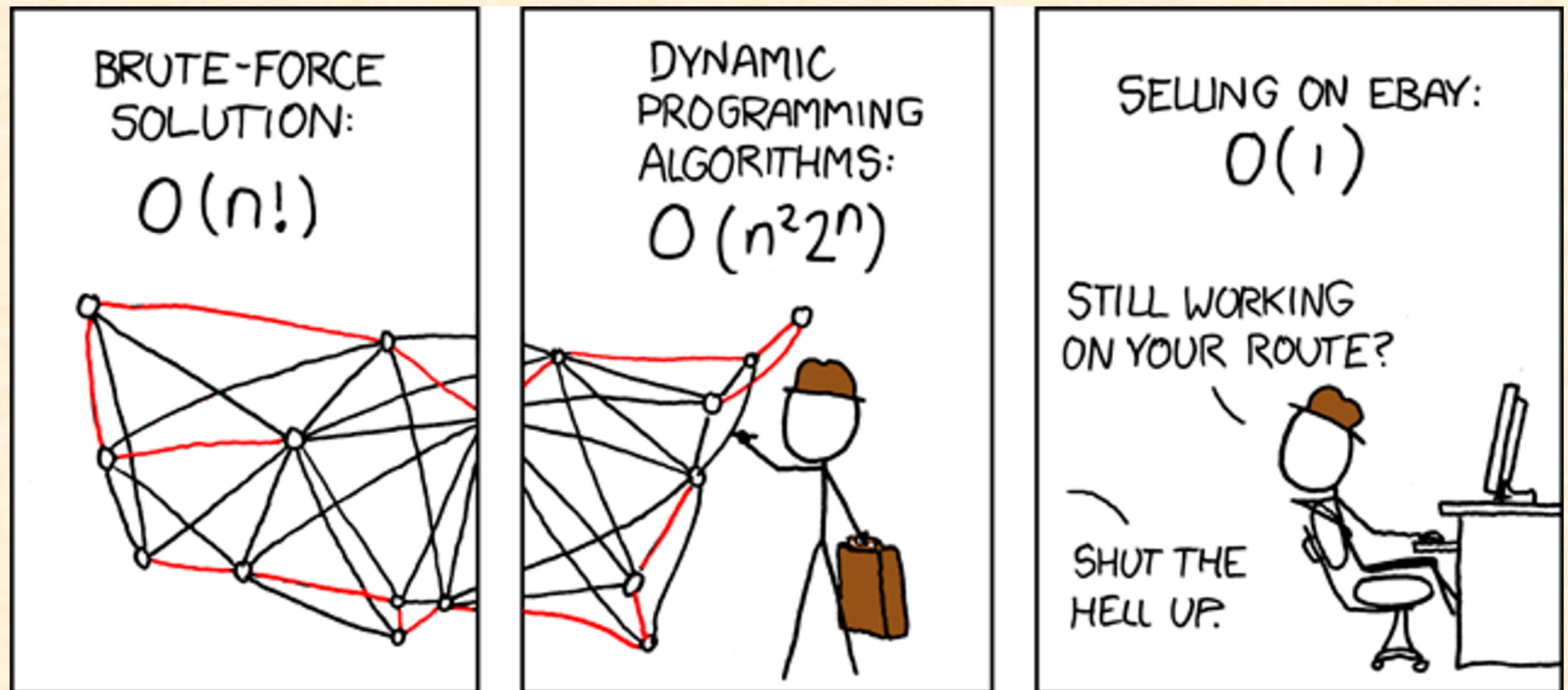
Generating Permutations

```
1  template <typename T>
2  void genPerms(vector<T> &path, size_t permLength) {
3      if (permLength == path.size()) {
4          // Do something with the path
5          return;
6      } // if
7      if (!promising(path, permLength))
8          return;
9      for (size_t i = permLength; i < path.size(); ++i) {
10         swap(path[permLength], path[i]);
11         genPerms(path, permLength + 1);
12         swap(path[permLength], path[i]);
13     } // for i
14 } // genPerms()
```

Optimal TSP With B&B

- Given n vertices, need to find best path out of $(n - 1)!$ options, use `genPerms()`
- Start with upper bound that is “infinity”, or better yet a fast calculation of a path that is guaranteed not shorter than optimal
- Use the upper bound to prune the rest of the search, lowering it every time a shorter, complete path is found
- Measure each partial solution, the path length of the first $1 \leq k$ points and estimate the cheapest cost to connect the remaining $n - k$ points, this is the lower bound
- Prune a partial solution if its lower bound exceeds the current upper bound
- If another complete path is shorter than the upper bound, save the path and replace the upper bound
- When the search is done, the current upper bound will be a shortest path

Branch and Bound & Traveling Salesperson Problem



<http://xkcd.com/399>

Traveling Salesperson Problem (TSP)



Data Structures & Algorithms

n Queens Demo



Data Structures & Algorithms

n Queens Implementation

- We know that:
 - Each row will have exactly one queen
 - Each column will have exactly one queen
 - Each diagonal will have at most one queen
- Don't model the chessboard as 2D array!
 - Instead, use 1D arrays of row position, column availability and diagonal availabilities
- To simplify the presentation, we will study for size 4x4

Implementing the Chessboard

First: We need to define an array to store the location of queens placed so far

`positionInRow`

1		Q		
3				Q
0	Q			
2			Q	

Implementing the Chessboard (column)

We need an array to keep track of the availability status of the column when we assign queens

Suppose that we
have placed two
queens

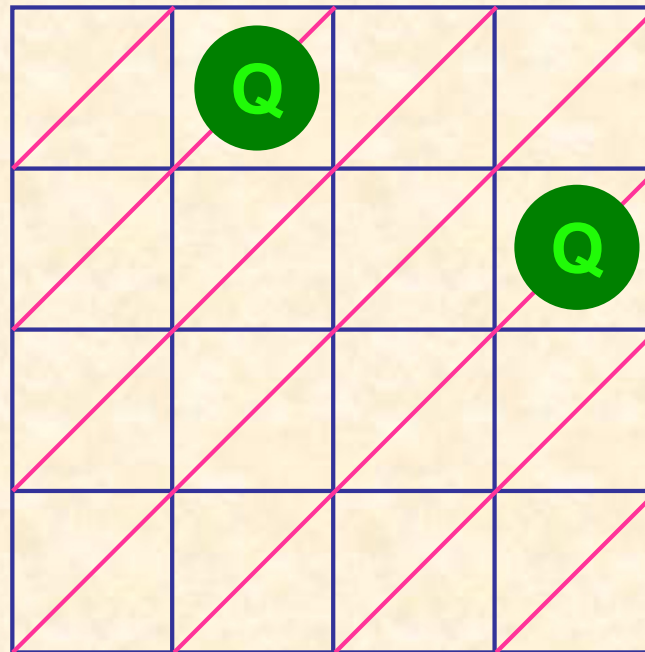
				
				
column	T	F	T	F

Implementing the Chessboard (L-Diag.)

With 7 left diagonals ($2 * N - 1$), keep track of available diagonals (indexed from top-left)

leftDiagonal

T
F
T
T
F
T
T



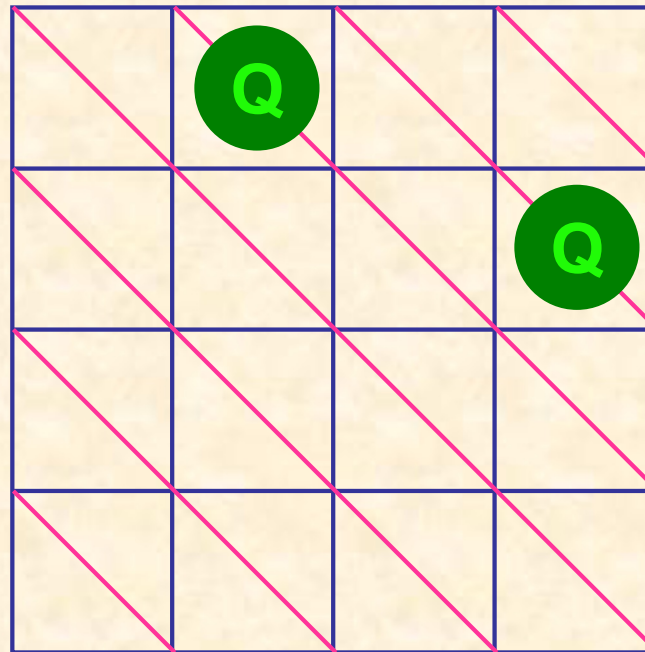
Diagonal Index = row + col

Implementing the Chessboard (R-Diag.)

Also 7 right diagonals (indexed from top-right)

rightDiagonal

T
F
F
T
T
T
T



$$\text{Diagonal Index} = (\text{row} - \text{col}) + (n - 1)$$

The promising() Function

```
1  bool NQueens::promising(uint32_t row, uint32_t col) {  
2      return    column[col] == AVAILABLE  
3              && leftDiagonal[row + col] == AVAILABLE  
4              && rightDiagonal[row - col + (n - 1)] == AVAILABLE;  
5  }  // promising()
```

The Recursive putQueen() Function

```
1  void NQueens::putQueen(uint32_t row) {
2      // Check for solution
3      if (row == n) {
4          cout << "Solution found" << endl;
5          return;
6      } // if
7      // Check every column in this row
8      for (uint32_t col = 0; col < n; ++col) {
9          if (promising(row, col)) {
10             // Make the move, and a recursive call to next move
11             positionInRow[row] = col;
12             column[col] = !AVAILABLE;
13             leftDiagonal[row + col] = !AVAILABLE;
14             rightDiagonal[row - col + (n - 1)] = !AVAILABLE;
15             putQueen(row + 1);
16
17             // Undo this move and thus backtrack
18             column[col] = AVAILABLE;
19             leftDiagonal[row + col] = AVAILABLE;
20             rightDiagonal[row - col + (n - 1)] = AVAILABLE;
21         } // if
22     } // for
23 } // putQueen()
```

} Place a piece @ (row,col)

} Remove piece @ (row,col)

Run n Queens Demo



Download

From a web browser:

`bit.ly/eecs281-nqueens-demo`

From a terminal:

```
% wget bit.ly/eecs281-nqueens-demo -O nqdemo.tgz
```

At the command line:

```
% tar xvzf nqdemo.tgz
```

```
% g++ -std=c++17 -O3 *.cpp -o nqueens
```

```
% ./nqueens
```

n Queens Demo



Data Structures & Algorithms